

Lecture 15 – Deep Learning: RNN

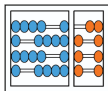
[ISLP 10.4–10.5]

MC886 – Machine Learning

Marcelo S. Reis ^{1,2}

[†] Departamento de Sistemas de Informação
Instituto de Computação, Universidade Estadual de Campinas

Campinas, April 22, 2026



**Instituto de
Computação**

UNIVERSIDADE ESTADUAL DE CAMPINAS



UNICAMP

Summary

Motivation: document classification
(and other sequential data)

Sequential models for document classification

Time series forecasting and autoregression

Summary of RNNs

Document classification

- ❖ It can be seen as predicting attributes of a document
- ❖ e.g., predicting whether a review is positive or negative
- ❖ That type of prediction is known as *sentiment analysis*
- ❖ We'll see now a study case with the IMDb¹

¹Internet Movie Database: developer.imdb.com/non-commercial-datasets.

Is this review 😊 or ☹ ?

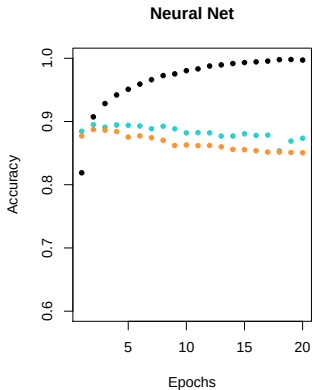
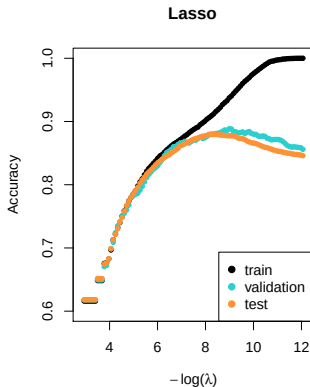
This has to be one of the worst films of the 1990s. When my friends & I were watching this film (being the target audience it was aimed at) we just sat & watched the first half an hour with our jaws touching the floor at how bad it really was. The rest of the time, everyone else in the theater just started talking to each other, leaving or generally crying into their popcorn...

- ❖ How to *featurize* the properties of that document?
- ❖ An alternative: the *bag-of-words* model

The bag-of-words model

- ❖ We define a dictionary of words (e.g., the 10 thousand most frequent words in English)
- ❖ Words less frequent as that are marked as “UNK” (unknown)
- ❖ For each review, we'll have a 10,000-dimensional binary vector (one dimension per word), which assigns whether a word is present (“1”) or absent (“0”) in the review
- ❖ Let's create such vectors for all 25K reviews in our training set and fit two models

LASSO and 2-hidden-layer neural network on IMDB



- ❖ Both training and test sets have 25K reviews each
- ❖ Each observation is a 10K-dimensional binary vector

Other shortcoming of bag-of-words

- ❖ It counts word frequency, ignoring its context in the document
- ❖ **Workaround 1:** the *bag-of-n-grams* model
 - Bag-of-2-grams: “blissfully long” vs. “blissfully short”
- ❖ **Workaround 2:** Treat the document as a sequence, so we can get the context of preceding words
- ❖ *This latter solution can be used in many applications*

Why? Many data sources are sequential!

- ❖ Documents (books, newspapers, tweets etc.)
- ❖ Time series of temperature, rainfall, etc.
- ❖ Financial time series
- ❖ Recorded speech, video, etc.

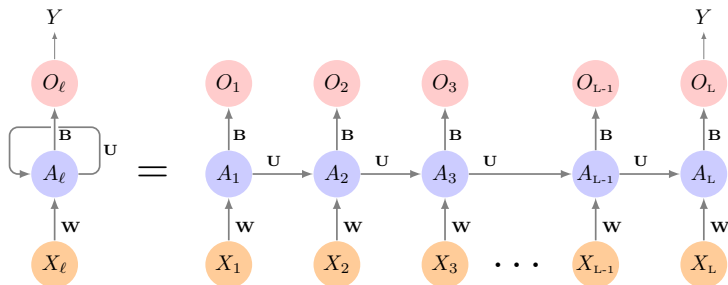
Sequential dataset

- ❖ An observation is a *sequence* $X = \{X_1, X_2, \dots, X_L\}$
- ❖ If X is a sequence of words, each X_ℓ represents a word
- ❖ The order of words has semantic meaning
- ❖ The response variable Y can be a category (e.g., the text is positive or negative) or a quantity (e.g., a rating)
- ❖ How do we use it to make predictions?

Recurrent neural network (RNN)

- ❖ Designed to take advantage of the nature of sequences (akin to CNN dealing with the spatial nature with images)
- ❖ Components of a RNN:
 - An input $X = \{X_1, X_2, \dots, X_L\}$, $X_\ell \in \mathbb{R}^p$
(a sequence of L *tokens*)
 - An output Y
(which might be a scalar value or a sequence)
 - A *hidden-layer sequence* $\{A_\ell\}_1^L = \{A_1, A_2, \dots, A_L\}$, $A_\ell \in \mathbb{R}^K$
 - An output layer O_L

Schematic of a simple RNN



- ❖ A_ℓ has K hidden unit activations $A_{\ell 1}, \dots, A_{\ell K}$
- ❖ $\mathbf{W}$: $K \times (p + 1)$ matrix of input layer weights
- ❖ $\mathbf{U}$: $K \times K$ matrix of hidden-to-hidden weights
- ❖ $\mathbf{B}$: $K + 1$ matrix of output layer weights

Input and hidden-to-hidden weights

- ❖ An activation $A_{\ell k}$ is computed by multiplying the input X_ℓ by an input weight vector $\mathbf{W}_k$ plus the activation vector $A_{\ell-1}$ multiplied by the hidden-to-hidden weight vector $\mathbf{U}_k$
- ❖ Therefore, $A_{\ell k}$ is computed as:

$$A_{\ell k} = g \left(w_{k0} + \sum_{j=1}^p w_{kj} X_{\ell j} + \sum_{s=1}^K u_{ks} A_{\ell-1, s} \right) \quad (10.16)$$

- ❖ g is an activation function (e.g., ReLU)
- ❖ A_0 is a zero-valued vector

Output weights

- ❖ The output layer O_L value is computed using a vector β :

$$O_L = \beta_0 + \sum_{k=1}^K \beta_k A_{Lk} \quad (10.17)$$

- ❖ Note that we can also compute O_ℓ for intermediate layers (useful if Y is also a sequence)
- ❖ The output above is suitable for regression; for classification problems, one should wrap it with a sigmoid function

Weight sharing

- ❖ Observe that we share $\mathbf{W}$, $\mathbf{U}$ and $\mathbf{B}$ among layers!
- ❖ There is no $\mathbf{W}^{(1)}$, $\mathbf{W}^{(2)}$, etc.
- ❖ It's something similar to convolution filters in CNNs

Learning of a RNN

- ❖ For a regression problem, we can use the squared error loss:

$$(y - O_L)^2, \quad (10.18)$$

where only O_L is employed ($O_1, O_2, \dots, O_{L-1}$ aren't used)

- ❖ With pairs (x_i, y_i) we minimize the sum of squares:

$$\sum_{i=1}^n (y_i - o_{iL})^2 = \sum_{i=1}^n \left(y_i - \left(\beta_0 + \sum_{k=1}^K \beta_k g(w_{k0} + \sum_{j=1}^p w_{kj} x_{iLj} + \sum_{s=1}^K u_{ks} a_{i,L-1,s}) \right) \right)^2 \quad (10.19)$$

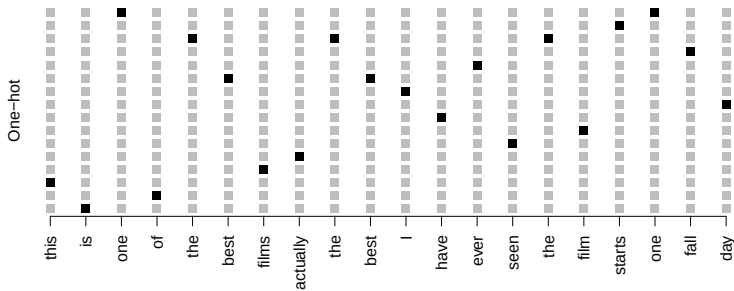
Sequential models for document classification

- ❖ Suppose we want to classify n reviews (i.e., n sequences of words) in positive or negative
- ❖ To put the review into the RNN, we first need to create a mathematical representation for each word in those texts
- ❖ One idea: to create a vector X with p entries, each one representing a word, defined as:

$$X_j = \begin{cases} 1, & \text{if word index } j \text{ is the one represented by the vector;} \\ 0, & \text{otherwise} \end{cases}$$

- ❖ Such a vector is known as **one-hot encoding**

Example of one-hot encoded vectors



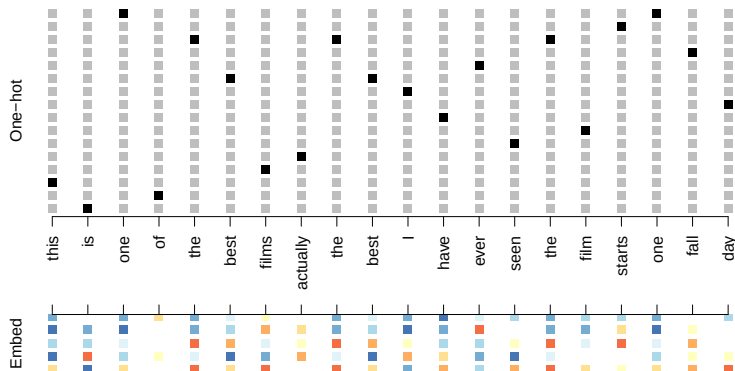
Limitations of one-hot encoding

- ❖ If we use each word in a dictionary, we'll have $p > 10000!!!$
- ❖ Data will be a large $n \times p$ sparse matrix
- ❖ **W** size is related to p
(might incur in overfitting and/or computational problems)
- ❖ How to solve it?

Workaround for the one-hot encoding

- ❖ Instead of representing a word with a huge binary vector, we'll employ much smaller m -dimensional, real-valued vectors of size ($m \approx 100s$)
- ❖ We convert the huge binary vector into the m -dimensional vector using a $m \times p$ matrix $\mathbf{E}$
- ❖ This technique is known as **word embedding**

One-hot encoded vs. embedded m -dimensional space



Here, $p = 16$ and $m = 5$

Learning word embeddings

- ❖ If we have a large corpus of labeled documents, $\mathbf{E}$ can be learned by training the RNN with an *embedding layer*
- ❖ Otherwise, we can use a pretrained $\mathbf{E}$ as the embedding layer
- ❖ Two widely used precomputed embeddings are:
 - word2vec
 - GloVe

Using the RNN on documents

- ❖ We limit each document to the last L words
- ❖ Documents shorter than L are padded with zeros upfront
- ❖ Thus, each document is a sequence of L vectors:

$$X = \{X_1, X_2, \dots, X_L\}, X_\ell \in \mathbb{R}^m$$

- ❖ Each X is processed sequentially from left to right

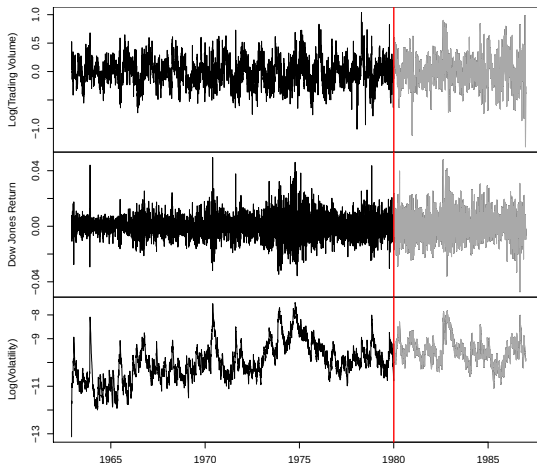
Long and Short Term Memory (LSTM)

- ❖ A generalization of the vanilla RNN
- ❖ Two tracks of hidden-layer activations are maintained
- ❖ When A_ℓ is computed, it gets input from back hidden units both close and further in time
- ❖ It improves the performance of the model, at the expense of added complexity
- ❖ In ISLP, for the **IMDb** dataset, the authors reported a test set accuracy of 76% and 87% for RNN and LSTM, respectively

Time series forecasting

- ❖ Example using the NYSE dataset
- ❖ Composed of three time series:
 - **Log trading volume:** fraction of all shares that are traded in a day (100-day moving average)
 - **Dow Jones return:** difference between the log of DJI Index on consecutive trading days
 - **Log volatility:** absolute values of daily price movements
- ❖ We want to predict trading volume using those time series

Example: trading statistics from NYSE

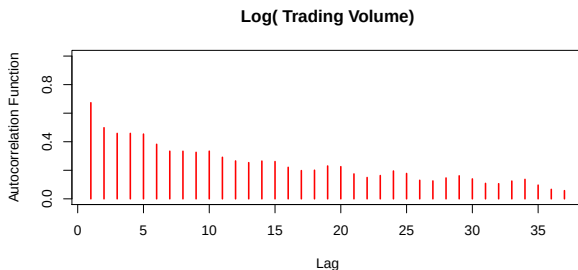


Red line divides training (left) and test (right) data

Autocorrelation

- ❖ Day-to-day observations are not independent to each other
- ❖ Nearby values tend to be similar to each other
- ❖ This phenomenon is known as “autocorrelation”
- ❖ For a log volume v_t on day t , we can compute the correlation between v_t and $v_{t-\ell}$, a lag of ℓ days apart

Autocorrelation for log trading volume



Idea: to use v_t also as an input!!!

RNN forecaster

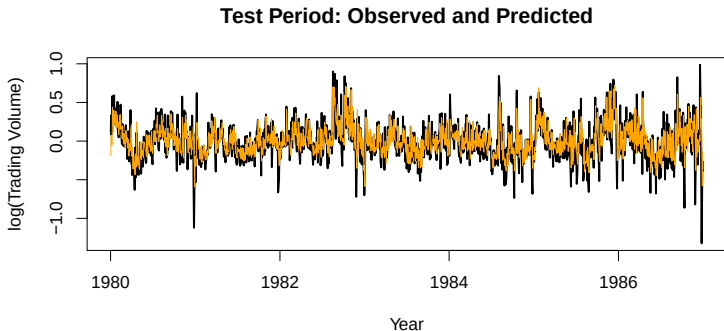
- ❖ NYSE dataset has 6,051 trading days
(number of triplets (v_t, r_t, z_t))
- ❖ Training data is obtained by defining segments of times series of size L , in the form:

$$X = \left\{ X_1 = \begin{bmatrix} v_{t-L} \\ r_{t-L} \\ z_{t-L} \end{bmatrix}, X_2 = \begin{bmatrix} v_{t-L+1} \\ r_{t-L+1} \\ z_{t-L+1} \end{bmatrix}, \dots, X_L = \begin{bmatrix} v_{t-1} \\ r_{t-1} \\ z_{t-1} \end{bmatrix} \right\},$$

$$\text{and } Y = v_t \quad (10.20)$$

- ❖ L is a hyperparameter that might be chosen by validation

Forecast of log trading volume on the NYSE test data



black: real time serie, **orange:** predicted time serie

Summary of RNNs

- ❖ We saw today the basic principles of RNNs
- ❖ There are many improvements on RNNs and/or architectures to replace them altogether
 - 1D CNN (treat sequence of embeddings as an image)
 - Bidirectional RNNs
(scan sequences in both directions)
 - Attention mechanisms
(weight the relevance of each part of the input)
 - Transformers (self-attention mechanism)